ClaimSafe Secure Campus Lost Property Claim and Collection System

Security 6000CMD and 6005CEM

Secure Application Development and Implementation

Ooi Ee Sheng

Coventry Student ID 15384832

30 October 2026

TABLE OF CONTENTS

1.0 Introduction and System Overview 1
1.1 System Overview 1
1.2 Objectives and Scope 1
2.0 Security Analysis and Design 2
2.1 Security Risks 2
2.2 Architecture and Access Control 2
2.3 Workflow Design 4
3.0 Security Implementation 5
3.1 Password Protection and Login Throttling 5
3.2 Multi-Factor Authentication and Recovery 5
3.3 Session Management and Authorisation 6
3.4 Input Validation and Output Protection 7
3.5 Secure Communication and Host Protection 7
3.6 Data Encryption and Recoverable Backups 8
3.7 Claim Approval and Collection 9
3.8 Logging Privacy and User Education 10
3.9 Security and Vulnerability Testing 11
4.0 Discussion 13
4.1 Evaluation and Findings 13
4.2 Limitations and Future Improvements 14
5.0 Conclusion 15
6.0 References 16
7.0 Appendix 17
7.1 Collection Code Consumption 17
7.2 Evidence Location and Versions 17
7.3 Acknowledgement of AI Assistance 17

LIST OF FIGURES

Figure 1 Implemented HTTPS architecture and trust boundaries 3
Figure 2 Claim and item state transitions 4
Figure 3 Collection confirmation in the initial prototype walkthrough 10
Figure 4 Administrator audit interface in the initial prototype walkthrough 10

LIST OF TABLES

Table 1 Security risks and selected controls 2
Table 2 Role and resource permissions 4
Table 3 Selected application and recovery results 11
Table 4 OWASP Top 10 2025 evidence and limitations 13

1.0 Introduction and System Overview

1.1 System Overview

ClaimSafe is a web application for managing campus lost property. Students can find an item in the public listings, submit evidence that it belongs to them and follow their claim. A property officer records the item when it comes into custody, assesses competing claims and confirms collection. Administrators can also review security activity and manage account activation. Anyone who registers through the public form receives a student account because staff accounts must be created locally by an operator.

The main problem is that information useful to a genuine owner can also help someone make a false claim. Publishing an engraving or the contents of a bag would allow another person to repeat those details. ClaimSafe therefore separates public descriptions from private identifying information. An officer compares a claimant's evidence with the withheld record before approving collection.

1.2 Objectives and Scope

ClaimSafe needs to keep private evidence available to the staff who assess it, while restricting what other students can see. It must also protect data in transit and prevent an approved item from being handed over twice. The assessment examines both the normal workflow and requests that try to get around these restrictions, with the audit log showing how selected actions were handled.

The system uses Django 5.2.18, cryptography 50.0.2 and SQLite on Windows. Its HTTPS profile uses Caddy 2.11.7 and Waitress 3.0.2 on loopback. The application baseline is commit aaf4b89; later report revisions do not change its security implementation. Testing uses fictional records. The scope is one campus, text evidence and collection in person, without payments, identity-document uploads or automatic proof of ownership.

2.0 Security Analysis and Design

2.1 Security Risks

A student who has signed in could change a claim identifier in the URL or send a request directly to a staff action. Someone registering could also submit fields that the public form never displays. A compromised officer account would expose more because officers can inspect identifying details and approve release. These risks make it necessary to check what each account can do after login and what authority a new account receives at registration.

Collection introduces a separate risk. A valid request could be repeated or sent concurrently, while an interrupted update could consume a code without completing the handover. Table 1 prioritises these risks according to their consequences for evidence and custody. These are qualitative design judgements rather than measured attack frequencies.

Table 1 Security risks and selected controls

Risk	Priority	Selected response
Cross-account evidence disclosure	High	Ownership-scoped retrieval and staff authorisation
Compromised staff credentials	High	Mandatory MFA, throttling and session invalidation
Interception or proxy bypass	High	Validated TLS and authenticated loopback backend
False or repeated collection	High	Approval states, expiring codes and atomic consumption
Malicious input or displayed content	High	Forms, parameterised queries, escaping and CSRF
Key loss or unauthorised file access	High	Private ACLs, encrypted snapshots and tested rotation
Missing or altered activity records	Moderate	Restricted audit views; external protection remains necessary

2.2 Architecture and Access Control

Students only need to retrieve their own claims, so limiting them to those records follows the principle of least privilege. ClaimSafe also checks permissions when an action is submitted, applying complete mediation to each request (Saltzer and Schroeder, 1975). This matters if an account loses access while its session is still open. The permission granted at sign-in may no longer apply.

The priorities also reflect how directly a request can affect a sensitive asset. Changing a claim identifier requires little knowledge of the implementation, while an unauthorised collection can affect physical custody. These receive high priority even though exposure is restricted to the local test environment. Audit alteration is rated moderate within the application threat model because ordinary users cannot edit those records; an operator with direct database access remains outside that protection. A real deployment would need to reassess likelihood against its users, network exposure and incident history.

Figure 1 shows the implemented HTTPS profile. Caddy terminates TLS before forwarding to the guarded WSGI application. The database and keys are outside the static web root. The browser cannot establish authority through a hidden field or a disabled button; the application obtains roles, ownership and state from server records.

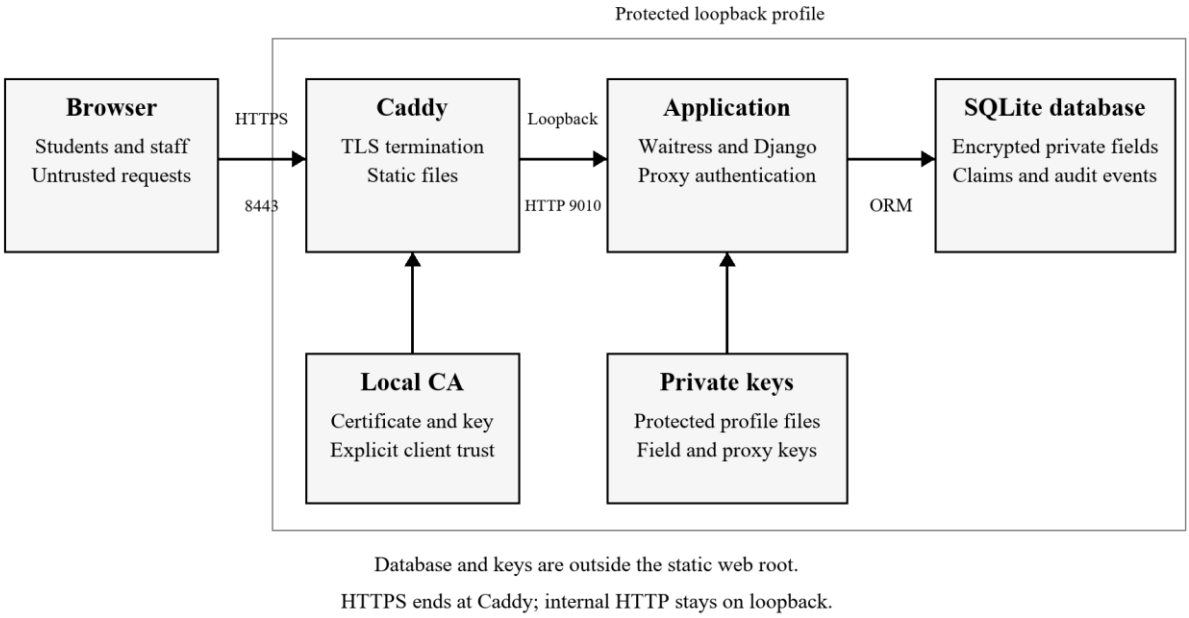


Figure 1 Implemented HTTPS architecture and trust boundaries

OWASP recommends denying access by default and validating permission on every request (OWASP Foundation, n.d.-a). ClaimSafe combines roles with record ownership: being a student permits claims, but does not justify reading another student's evidence. Table 2 summarises the permitted operations. Staff access also requires completed MFA.

Table 2 Role and resource permissions

Capability	Student	Officer	Administrator
Browse public listings	Yes	Yes	Yes
Read private claims	Own	Operational claims	Operational claims
Read withheld item details	No	Yes	Yes
Review and confirm collection	No	Other people's claims	Other people's claims
View system audit and manage activation	No	No	Yes

Officers can inspect evidence across the campus because they need it to assess claims. The application decrypts that evidence for them, so the server operator remains trusted. A database copy alone exposes less because the fields are encrypted, but an operator who controls both the database and the keys can read or alter them.

2.3 Workflow Design

Figure 2 separates approval from collection. Approval reserves the item and closes competing pending claims without identifying the successful claimant. Collection requires an approved claim, its current code and an officer's confirmation. Self-review and self-handover are rejected so that an officer cannot authorise their own release.

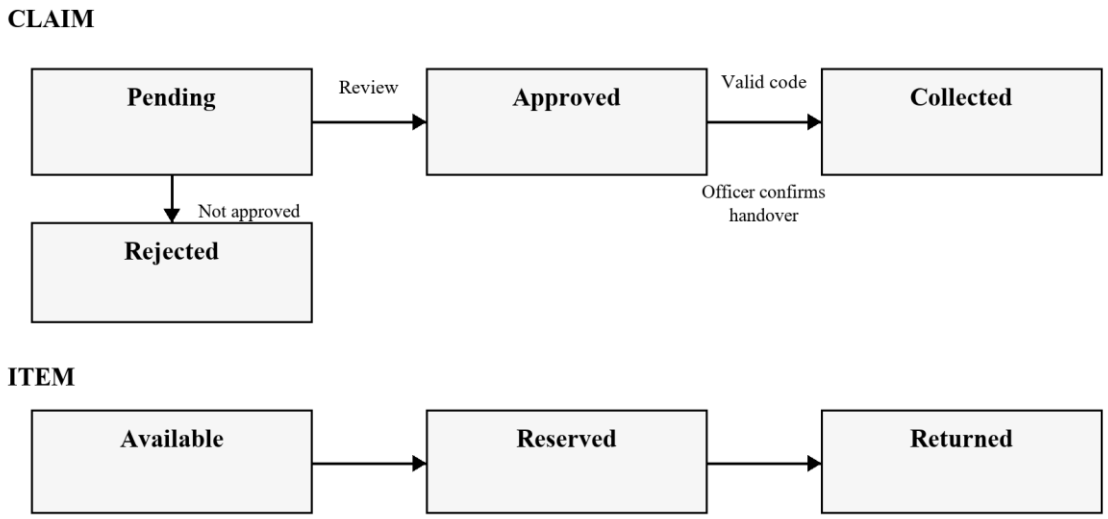


Figure 2 Claim and item state transitions

The server rejects invalid or repeated transitions. The officer still checks who is collecting the item because a valid code only authorises the collection step.

3.0 Security Implementation

3.1 Password Protection and Login Throttling

Registration uses an explicit field allowlist and assigns the student role server-side. A test submits administrator parameters and verifies that the resulting account remains a student without elevated framework flags. Privileged provisioning uses a management command rather than public registration.

Django stores passwords as salted PBKDF2 hashes (Django Software Foundation, n.d.-b). ClaimSafe originally required twelve characters. The minimum was raised to fifteen because students can use a password as their only factor, and fifteen is NIST's minimum for that case (NIST, 2025). The form also rejects common passwords, all-numeric passwords and values too similar to account details.

Login counters persist in the database. Within fifteen minutes, the account limit permits five attempts and the network limit permits thirty. The counters use identifiers derived with HMAC so they can track repeated attempts without storing the submitted usernames and addresses in those rows. Once the network is blocked, further requests from it cannot create new account counters. A legitimate user can also hit a limit, so the error explains when to retry.

3.2 Multi-Factor Authentication and Recovery

Password-only staff sessions cannot open private records or operational pages. Officers and administrators must enrol a time-based authenticator; students may enable one voluntarily. Setup requires the current password, generates its QR locally and binds the pending secret to the initiating browser for ten minutes. The database stores the secret encrypted and the setup handle as a digest.

PyOTP generates six-digit codes in thirty-second steps. The implementation accepts the current step and one adjacent step on either side to accommodate clock differences. A conditional counter update consumes each accepted step once, including concurrent requests. This addresses PyOTP's guidance to protect secrets, use HTTPS, throttle attempts and reject replay (PyOTP, n.d.).

Each of the ten recovery codes contains 128 random bits. The user sees the codes when they are created, and the database keeps only digests tied to that user. An unused code can complete MFA or authorise full recovery with a new password. Full recovery resets authenticator

enrolment and revokes saved codes and existing sessions, while any account suspension still applies. If a user loses every factor, an operator has to verify their identity before restoring access.

The replay check in core/account_security.py uses the account's current version and last accepted counter.

```
accepted = AccountSecurity.objects.filter(
    user=user, version=state.version,
    last_totp_counter__lt=counter,
).update(last_totp_counter=counter) == 1
```

Checking a code and then recording it in separate operations would leave time for another request to reuse it. Here, success depends on one conditional update inside the transaction. The concurrent test starts two database connections together and expects exactly one acceptance. The HTTPS sequence also checks the practical consequence: a second browser remains unable to open the staff desk after submitting the consumed code.

3.3 Session Management and Authorisation

Django rotates sessions during authentication, and the application enforces a thirty-minute inactivity boundary and eight-hour absolute lifetime. Sensitive security changes recheck the current account state. Password changes retain the current valid session through update_session_auth_hash while other sessions become invalid, following Django's documented behaviour (Django Software Foundation, n.d.-c).

An MFA proof carries the account's credential version. A security reset increments that version, so a proof issued before the reset is no longer accepted. The application also checks whether the account has been deactivated, which prevents someone from continuing to use an open session after access is withdrawn.

Student claim retrieval includes the authenticated claimant in its lookup. Requesting another student's identifier returns a generic 404 without the evidence. Officers cannot open administrator audit or activation functions. State-changing routes require POST and repeat permission checks; hiding a control in the interface is not treated as an access restriction.

The ownership filter in core/views.py narrows the records before the application retrieves the requested claim.

```
if not request.user.is_officer:
    claims = claims.filter(claimant=request.user)
claim = claims.filter(pk=pk).first()
```

The test requests a claim that exists and belongs to another student, then checks that the response is 404 and the evidence is absent. A separate test checks that the owner can open their own claim. Together these results show that the response depends on ownership, even when the requester knows a valid identifier.

3.4 Input Validation and Output Protection

Django forms reject malformed or excessive input before workflow services act. Claim evidence requires twenty to 2,000 characters, public descriptions are limited to 300, and future found dates are rejected. Service checks repeat important constraints because requests need not originate from the displayed form.

ClaimSafe passes submitted values through Django's ORM so the database treats them as values within the query. Valid text can still contain markup, so the templates keep automatic escaping and quote attributes when displaying it. Disabling escaping or constructing a query unsafely would remove the relevant protection (Django Software Foundation, n.d.-d).

CSRF middleware checks form tokens and origins. Enforced-CSRF tests reject a missing token, while selected HTTPS requests also rejected a foreign origin with a valid token. A script-like evidence submission was displayed as escaped text. CSP disallows scripts and embedding, and generic errors omit tracebacks. The tests establish these behaviours on examined paths, not immunity against every payload.

3.5 Secure Communication and Host Protection

The HTTPS profile redirects HTTP on port 8080 to localhost:8443. Caddy presents a local certificate; Waitress listens privately on 127.0.0.1:9010. All configured listeners remain on loopback. A random proxy credential is required before Django receives a request and is removed from the environment before application error handling. Forged forwarding headers alone therefore cannot bypass the backend boundary.

Production cookies are Secure, HttpOnly and SameSite=Lax, using the __Host- prefix and Path=/. Sensitive responses return no-store. Host validation runs on every request, while the proxy rejects unexpected hostnames. An initial empty success response for an unexpected Host was corrected; retesting returned 421 instead.

Windows profile ACLs permit the current user and SYSTEM. Database, key and TLS directories are separate from served static files, and the verified processes did not hold an administrator token. This is scoped local hardening, not a dedicated service account.

TLS protects the connection from the browser to Caddy, and the HTTP hop from Caddy to Waitress stays on loopback. A client still needs to verify the certificate's chain, hostname and validity period. Caddy's internal CA needs to be trusted explicitly by the client (Caddy, n.d.). The assessment succeeded when it supplied the project CA and used the correct hostname, and failed with the wrong hostname or default public trust. The ordinary browser also rejected the certificate because trust had yet to be configured. The warning was left in place and the system trust store was left as configured.

3.6 Data Encryption and Recoverable Backups

Identifying details, claim evidence, review messages and authenticator secrets are encrypted before storage. Fernet provides authenticated encryption, using AES-128-CBC with HMAC-SHA256. Its library also exposes token creation time, and anyone holding the key can decrypt or forge tokens (Python Cryptographic Authority, n.d.). Passwords remain hashes because the service never needs to recover their original values.

Field keys are generated independently of Django's signing key. Setup refuses replacement keys beside an existing database with missing secrets, avoiding silent loss of readable evidence. Keys, credentials and databases are excluded from Git.

Offline snapshots encrypt the database and matching secrets together under an independent backup key. Restore requires a new destination and verifies authentication, integrity and decryption before exposing the profile. It revokes sessions, collection grants and recovery codes, clears pending MFA and regenerates signing/proxy secrets. Later account changes and physical handovers still require reconciliation because an older snapshot cannot contain them.

Rotation starts with a verified snapshot. The new primary key is saved alongside the old keys before the encrypted fields are rewritten in a transaction, so existing records remain readable during the change. The fallback keys can be retired once the new key alone decrypts every field. MultiFernet preserves token timestamps through rotation (Python Cryptographic Authority, n.d.). A shared profile lease prevents the server and maintenance tools from using the profile at the same time.

Recovery was tested against a separate copy so that a failed exercise would leave the original database intact. Tests checked that a wrong key or a tampered archive is rejected before a usable destination appears. The rotation rollback test deliberately fails final verification and then checks that the original ciphertext is still readable with the retained key ring. Off-device storage is still an operator task because a backup is unusable if its independent key is lost.

3.7 Claim Approval and Collection

Approval conditionally changes an available item to reserved inside a transaction. A database constraint permits only one approved or collected claim per item. If a competing request has already changed the state, approval stops instead of overwriting it.

Only the approved claimant can generate a collection code. It contains 192 bits of random input, expires after fifteen minutes and is stored as a SHA-256 digest. Reissuing replaces the previous digest. At handover, the officer supplies the code and confirms the claimant. Consumption, claim/item transitions and the completion event commit together; failure rolls back the operation.

Django documents that SQLite does not implement SELECT FOR UPDATE row locks (Django Software Foundation, n.d.-a). ClaimSafe therefore uses IMMEDIATE transactions and conditional writes. In a test with two database connections trying to consume the same code, one collection succeeded, the other failed, and the audit log recorded a single completion. Appendix 7.1 shows the condition the update has to satisfy.

Figure 3 records the initial browser walkthrough on 25 October. It illustrates the collection interface; current replay and concurrency behaviour are supported separately by regression and HTTPS evidence.

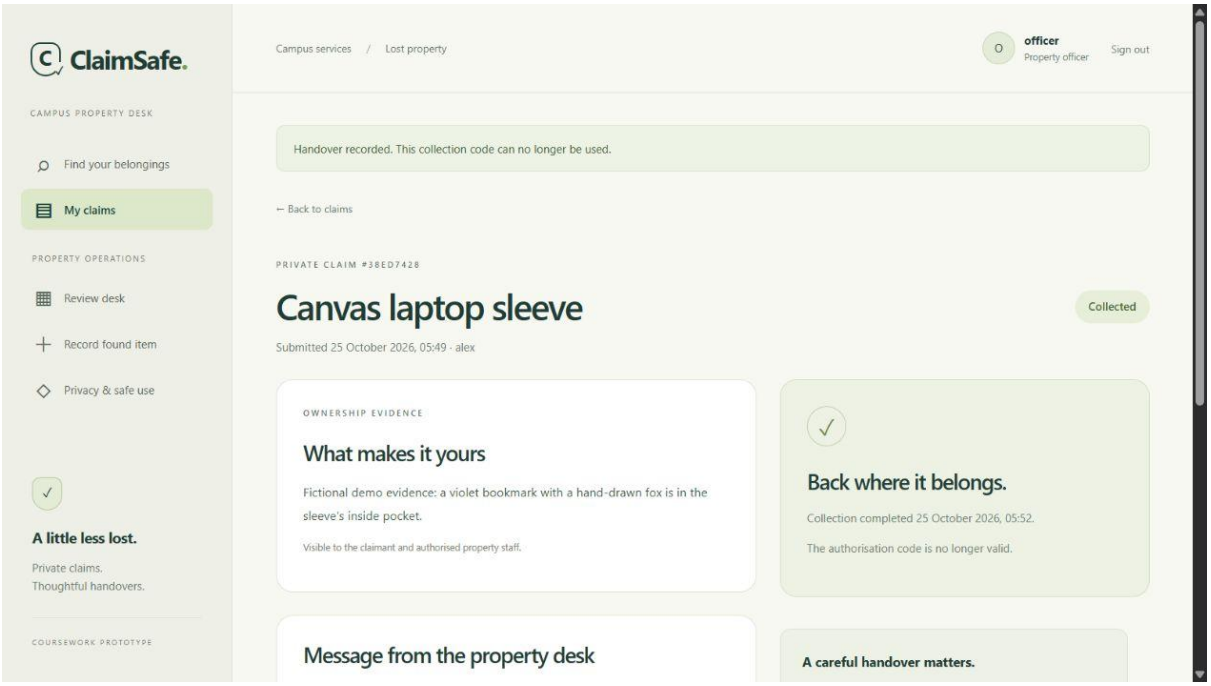


Figure 3 Collection confirmation in the initial prototype walkthrough

3.8 Logging Privacy and User Education

Audit events record actor, action, target, outcome and timestamp for login failures, denied access, evidence views, review decisions, security changes and handovers. Students see their own history; administrators see system activity, filters and counts. Figure 4 shows the initial audit interface, whose captured totals are demonstration records rather than an attack-rate measurement.

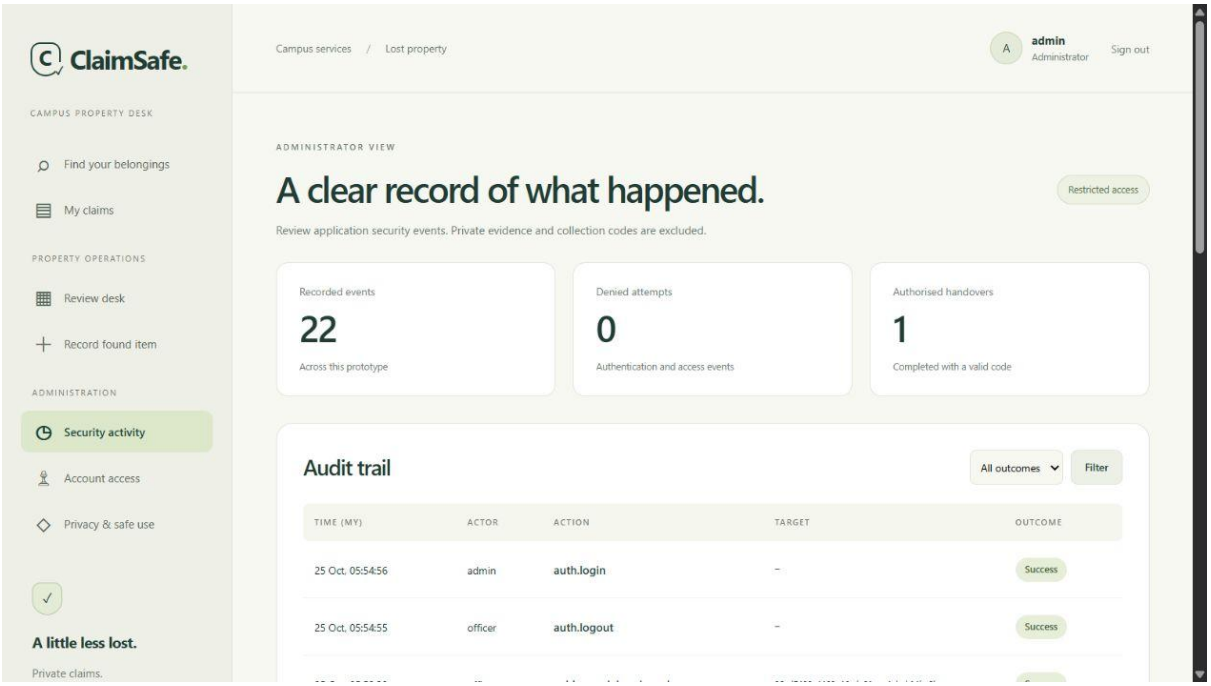


Figure 4 Administrator audit interface in the initial prototype walkthrough

Logs exclude passwords, evidence, authenticator secrets and submitted codes. OWASP advises excluding authentication secrets and other sensitive data from application logs (OWASP Foundation, n.d.-b). Restricted dashboard access helps operational review, although a database administrator can still alter the underlying audit records.

The privacy page explains collection purposes, audiences, recovery and encryption limits. Forms warn against identity numbers, device PINs and payment information. These choices reflect notice, security, disclosure and retention concerns described by Malaysia's Ministry of National Unity (n.d.). Applicability and operational obligations require assessment for the actual institution; fictional testing does not establish legal compliance. Expired sessions, counters and pending enrolments can be pruned, but general retention and deletion procedures remain unresolved.

3.9 Security and Vulnerability Testing

The assessment combined application tests with selected requests against an isolated HTTPS profile using fictional accounts. On 30 October, 83 automated tests passed in 41.767 seconds, the recorded duration for that run. A further 19 HTTPS account checks passed with certificate validation. Table 3 gives selected outcomes, with the setup and results available in the source and redacted evidence.

Table 3 Selected application and recovery results

Check	Observed result
Cross-account claim request	404; private evidence absent
Privileged registration parameters	Student role retained
Missing CSRF token or foreign origin	403
Password-only staff session	Redirected to MFA; desk inaccessible
Reused TOTP or recovery code	Rejected; second session remained blocked
Concurrent TOTP or collection requests	One successful consumption
Password change or account recovery	Older sessions invalidated
Wrong backup key or modified archive	Restore rejected without destination
Isolated restore and rotation	24 encrypted values preserved

The backup exercise on 26 October restored a real fictional database into a separate profile, rotated its 24 encrypted values and retired the old key. Fresh application processes still decrypted every value. Source database and key hashes were unchanged. Unit tests additionally cover failed rotation rollback, existing-output refusal and cross-process leases.

The earlier assessment at commit 5339995 included anonymous OWASP ZAP 2.17.0 scanning of seventeen URLs with five selected active rules. It returned three informational alert types and no low, medium or high alerts. Review distinguished cacheable static files and escaped attribute text from sensitive responses. SSLyze 6.3.1 accepted TLS 1.2/1.3 and rejected older protocols; the supplied CA validated, while public trust stores did not. These are revision-specific results, not a scan of every later MFA endpoint.

The installed dependency audit initially returned seven records representing four distinct advisories for cryptography 46.0.7. Updating to 50.0.2 preserved readable evidence. The 30 October pip-audit run examined twelve installed application packages and returned no known advisories. Bandit's latest code assessment retained twelve findings: empty secret-reset values, fixed no-shell maintenance calls and a checksum-verified HTTPS downloader. Each was reviewed rather than silently suppressed. No direct Fernet exploit was demonstrated.

The selected request checks varied credentials, roles, identifiers, origins and supplied headers independently. Earlier collection probing returned a redirect for the first handover and 409 for its replay, with one completion event in storage. Forged forwarding addresses did not avoid network throttling. Public requests for the key file, Git metadata and source paths returned 404. These observations connect the controls to attempted misuse rather than relying on the normal interface alone. Retained manifests distinguish these earlier cases from the refreshed account checks.

4.0 Discussion

4.1 Evaluation and Findings

The account and collection tests show why the application needs several checks after a user signs in. An authenticated student was still unable to retrieve someone else's evidence, and an officer with only a password could not open operational records. Once a claim was approved, the server allowed one valid handover and rejected reuse of the code. This kept the legitimate workflow available while enforcing the account and collection restrictions.

The assessment used OWASP Top 10 2025 to organise access, configuration, supply-chain, cryptographic, injection, design, authentication, integrity, logging and exceptional-condition concerns (OWASP Foundation, 2025). Coverage varies. Request tests directly examine access and authentication, while dependency tools identify reported package weaknesses. An anonymous scan cannot establish the security of authenticated workflows, and reviewed static warnings are not proof that all code is safe.

Table 4 connects each category to the available evidence and a remaining boundary. This avoids treating a list of installed protections as evidence that an entire vulnerability category has been eliminated.

Table 4 OWASP Top 10 2025 evidence and limitations

Category	Evidence	Remaining boundary
A01 Broken Access Control	Ownership, role and direct-request checks	Broad officer access remains trusted
A02 Security Misconfiguration	Host rejection, private ACLs, proxy checks	Dedicated service identity not provisioned
A03 Software Supply Chain Failures	Dependency update and repeat audit	Operating system and all tools not audited
A04 Cryptographic Failures	TLS validation, encryption and rotation	Host operator holds decryption capability
A05 Injection	Query, markup and selected scanner probes	Inputs and payload coverage are bounded
A06 Insecure Design	Approval states and concurrent collection tests	Physical ownership still needs judgement
A07 Authentication Failures	MFA replay, recovery and session checks	TOTP remains susceptible to phishing

Category	Evidence	Remaining boundary
A08 Software or Data Integrity Failures	Authenticated snapshots and restore verification	Later custody/account changes need reconciliation
A09 Logging and Alerting Failures	Restricted audit and secret-exclusion checks	No independent log sink or alerting
A10 Mishandling of Exceptional Conditions	Rollback, corrupt-archive and error tests	Resource-exhaustion coverage remains limited

Two problems appeared during verification on Windows. SQLite connections stayed open after transactions and prevented file cleanup, which worked once the connections were closed explicitly. Windows also created directory grants that had to be removed before the launcher accepted the profile permissions. Both problems affected the local operating environment and only became visible when the recovery and launch procedures were exercised.

4.2 Limitations and Future Improvements

Staff judgement remains necessary: someone who already knows identifying details may submit a convincing claim. Operational staff can inspect campus-wide evidence, while the host operator controls decryption and audit storage. TOTP improves password protection but remains susceptible to phishing and depends on clock accuracy. Recovery codes require private storage, and losing every code/device needs an agreed identity-review process.

The immediate operational priorities are browser CA trust, the user's own factor enrolment and separate off-device custody of backup archives and keys. Public hosting and a dedicated service identity require an operator-selected environment. Retention/deletion procedures and an independently protected audit destination should follow. A future assessment should extend authenticated scanning and test changes to database or hosting configuration. These improvements address identified boundaries; adding unrelated features would not strengthen the existing evidence.

5.0 Conclusion

ClaimSafe provides a working lost-property workflow with role and ownership restrictions, staff MFA, protected communication, encrypted evidence and controlled collection. Testing supports the specified access, replay, concurrency and recovery behaviour within the examined Windows/SQLite environment.

Private profile permissions restrict access to local files, and the guarded loopback backend limits how requests reach the application. Together with TLS, these controls address the secure-environment objective. The tests of legitimate and invalid requests show how the application enforces the security rules during a working handover. Before real campus use, the institution would still need to configure browser trust, provision user-owned factors and agree how to operate the service.

6.0 References

Caddy (n.d.) Automatic HTTPS. Available at: https://caddyserver.com/docs/automatic-https (Accessed: 30 October 2026).

Django Software Foundation (n.d.-a) Databases. Available at: https://docs.djangoproject.com/en/5.2/ref/databases/ (Accessed: 30 October 2026).

Django Software Foundation (n.d.-b) Password management in Django. Available at: https://docs.djangoproject.com/en/5.2/topics/auth/passwords/ (Accessed: 30 October 2026).

Django Software Foundation (n.d.-c) Using the Django authentication system. Available at: https://docs.djangoproject.com/en/5.2/topics/auth/default/ (Accessed: 30 October 2026).

Django Software Foundation (n.d.-d) Security in Django. Available at: https://docs.djangoproject.com/en/5.2/topics/security/ (Accessed: 30 October 2026).

Ministry of National Unity (n.d.) Personal Data Protection. Available at: https://www.perpaduan.gov.my/index.php/en/services/other-services/e-participation/personal-data-protection (Accessed: 30 October 2026).

National Institute of Standards and Technology (NIST) (2025) Digital Identity Guidelines Authentication and Authenticator Management. SP 800-63B-4. Available at: https://pages.nist.gov/800-63-4/sp800-63b.html (Accessed: 30 October 2026).

OWASP Foundation (2025) OWASP Top 10 2025. Available at: https://top10.owasp.org/2025/ (Accessed: 30 October 2026).

OWASP Foundation (n.d.-a) Authorization Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (Accessed: 30 October 2026).

OWASP Foundation (n.d.-b) Logging Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html (Accessed: 30 October 2026).

PyOTP (n.d.) The Python One-Time Password Library. Available at: https://pyotp.readthedocs.io/en/stable/ (Accessed: 30 October 2026).

Python Cryptographic Authority (n.d.) Fernet symmetric encryption. Available at: https://cryptography.io/en/stable/fernet/ (Accessed: 30 October 2026).

Saltzer, J.H. and Schroeder, M.D. (1975) The protection of information in computer systems. Proceedings of the IEEE, 63(9), pp. 1278-1308. Available at: https://web.mit.edu/Saltzer/www/publications/protection/Basic.html (Accessed: 30 October 2026).

7.0 Appendix

7.1 Collection Code Consumption

The following extract is from core/services.py at the application baseline. It consumes only a matching, unused and unexpired authorisation. The surrounding transaction also updates custody states and the audit event.

```
changed = CollectionGrant.objects.filter(
    claim=claim, digest=hashlib.sha256(token.encode()).hexdigest(),
    used_at__isnull=True, expires_at__gt=now,
).update(used_at=now)
if changed != 1:
    raise WorkflowError(
        "The collection code is invalid, expired or already used.",
        400,
    )
```

7.2 Evidence Location and Versions

The local repository retains source, tests and redacted results. The 26 October web/TLS assessment is under docs/security-evidence/; account and data-recovery evidence is under its account-recovery subdirectory. Final 30 October regression, dependency and HTTPS observations are under report-final-2026-10-30. Source fingerprints identify assessed files. Raw credentials, codes, keys and databases remain outside version control.

7.3 Acknowledgement of AI Assistance

OpenAI Codex assisted with planning, application implementation, test automation, source finding, technical explanation, report drafting and document formatting. The assistance included code and substantive writing. This acknowledgement describes the assistance used and does not replace the official Generative AI Declaration Form.